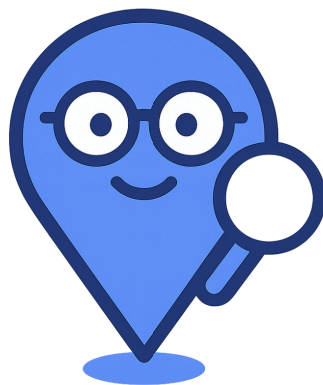


가천 X 세종

연합 학술제

구현 파트 보고서

본 가이드라인은 작성 시 반드시 준수해야 할 최소 기준입니다.
추가적인 내용은 자유롭게 포함할 수 있으나, 아래 항목들을 명확한 사유 없이 누락할 경우,
평가에 불이익이 반영될 수 있습니다.



timebillgates

시간 빌게이츠

팀장	안성현	팀원	이 찬
팀원	임지수	팀원	정은우

2025.08.22

목차

1. Problem Definition & User Analysis	3
a. Problem Definition & User Analysis	3
b. MVP (Minimum Viable Product)	4
c. 목표	4
2. Technology Stack Selection	5
a. Stack Selection	5
b. Reason for selection	6
3. Requirement Analysis	7
a. 기능적 요인 (Functional Requirements)	7
b. 비기능적 요인 (Non-functional Requirements)	7
4. Service Architecture Design	8
a. 전체 파이프라인	8
b. 주요 설계 포인트	8
5. Development Process Planning	9
a. 깃 전략	9
b. 협업 방식	9
c. 깃허브 디렉토리 구조	10
6. Data & API Specification	11
a. ERD, 테이블 간 관계	11
b. SQL trigger	12
c. API 명세	13
7. Security & Deployment Setup	14
8. AI Feature Specification	15
a. 시스템 개요	15
b. 핵심 기능	15
c. 시스템 아키텍처 및 데이터 인터페이스	16
d. 프로젝트 배경 및 동기	16
e. 문제 해결을 위한 개발 전략	17
f. 모델 개발 상세 과정	17
g. 성능 검증	18
9. Wireframe & User Flow Design	21
a. Ui 설계도	21
b. 웹 서비스 로고 및 캐릭터, 컬러 테마	22
10. 웹 서비스 실행 가이드	23
a. 깃허브 링크	23
b. 참고 문서	23

▲ 가천대의 넓은 캠퍼스 내 멀리 분포되어 있는 학습 라운지

- 또한 가천대학교는 넓은 캠퍼스 특성상 팀 프로젝트 회의 공간이 여러 건물에 분산되어 있으며, 특정 장소에 도착하더라도 인원이 많아 사용이 불가능한 경우가 발생한다. 이 경우 학생들은 다시 다른 공간을 찾아야 하는데, 이는 시간과 노력을 소모하게 만든다.
- 이러한 문제들을 종합적으로 고려했을 때, 단순히 예약 기능에 의존하는 것이 아니라 **실시간으로 사람과 사물을 인식하여 공간의 실제 사용 현황을 파악하고, 여석 정보를 제공하는 시스템**이 필요하다고 판단하였다. 따라서 본 프로젝트는 이러한 문제 해결을 목표로 서비스를 구체화하였다.

● User Analysis

- 학생(개인): 실시간 좌석 정보 기준으로 가장 가까운 공용공간의 빈 좌석을 선호한다.
- 학생(팀): 위 기준에 더해 여러 명이 함께 앉을 수 있는 회의 공간을 필요로 한다.

● MVP (Minimum Viable Product) : 실시간 혼잡도 기반 장소 추천

- **혼잡도 기반 추천**: 실시간 혼잡도(점유율)가 낮은 순으로 정렬해 추천 (*동률 시 거리순*).
- **인원수 기준 추천**: 데이터베이스(DB)에 저장된 최대 수용 인원과 혼잡도를 고려해, 요청 인원을 수용할 수 있는 공간만 필터링하여 추천.
- **위치 기반 추천**: 사용자의 현재 위치와의 거리가 짧은 순으로 정렬해 추천 (*동률 시 혼잡도순*).

● 목표

- 본 프로젝트는 현 시점에서 교내 CCTV 스트림 접근 권한이 없다는 제약을 전제로 한다. 이에 따라 영상 입력 계층을 **RTSP 표준(IP CCTV 표준 스트리밍 제어 프로토콜)**으로 통일하고 파이프라인을 벤더 비종속적으로 구성하였다. **향후 권한을 획득할 경우 카메라 RTSP 주소만 교체**하면 추가 개발 없이 **동일 파이프라인으로 즉시 서비스 가동**이 가능하도록 하는 것이 목표이다.
- 또한, 카메라가 **교내 공용 네트워크**에 속해 있어 실험 단계에서는 **외부 서버와의 직접 연동(배포) 테스트를 수행하지 못했으나**, 향후 CCTV 스트림 접근 권한 및 네트워크 허용을 확보하면 **영상 소스(URL) 교체만으로 동일 아키텍처에 즉시 연동**이 가능하다.

■ Technology Stack Selection

● Stack Selection



-
- **스트리밍:** Tapo 카메라 (TC60)이용 / RTSP 프로토콜
- **CV/추론:** YOLOv5(Custom) + Python(OpenCV), Polygon(ROI) 후처리.
- **백엔드 API:** Flask(웹 프레임워크) + Gunicorn(애플리케이션 서버).
- **DB:** Supabase(Postgres, Realtime, SQL Trigger/Function).
- **배포/인프라:** GCP VM(리눅스) + Nginx(리버스 프록시, TLS).
- **프론트엔드:** React(대시보드)

● Reason for selection

- **RTSP (IP CCTV 표준 스트리밍 제어 프로토콜)**
캠퍼스 내 CCTV 영상을 활용한다는 서비스 가정을 충족하며, 표준화된 스트리밍 방식으로 안정적인 영상 전송이 가능하다.
- **YOLOv5 (AI 모델)**
경량화되어 실시간 추론이 가능하며, 성숙한 커뮤니티와 다양한 튜토리얼이 존재하여 개발 효율성이 높다. 또한 기존 파이프라인과 호환성이 뛰어나고 모델 튜닝이 용이하다.
- **Supabase (Database)**
관리형 PostgreSQL을 기반으로 하여 백업, 모니터링, 자동 패치 기능을 제공함으로써 운영 부담을 최소화할 수 있다. 또한 Realtime 기능을 통해 좌석 점유 상태 변화가 발생하면 자동으로 화면에 반영되어 별도의 새로고침 없이 실시간 데이터 갱신이 가능하다.
- **GCP VM (서버 인프라)**
비용 예측과 제어가 용이하며, 안정적으로 Flask 기반 백엔드와 AI 추론 모듈을 배포할 수 있는 환경을 제공한다.
- **Flask (웹 프레임워크)**
팀 내에서 익숙한 기술 스택으로, 경량 WSGI 기반 프레임워크이기 때문에 빠른 입출력(I/O) 처리와 신속한 개발이 가능하다. 또 React 와 함께 개발하기에 적합하다.

- **Gunicorn (WSGI 서버)**

멀티 워커 및 스레드 방식을 통해 동시성을 확보하여 프로덕션 환경에서 안정적인 서빙이 가능하다. 이를 통해 Flask 단점을 보완하고 확장성을 강화하였다.

- **Nginx (Reverse Proxy 서버)**

정적 파일 서빙과 버퍼링 기능을 제공하여 서버 부하를 줄이고, 안정적인 요청 분산 및 보안 기능을 지원한다.

- **React (프론트엔드 프레임워크)**

컴포넌트 기반 구조를 통해 개발 효율성이 높으며, 팀원들이 강의와 프로젝트 경험을 통해 이미 익숙한 언어라는 점에서 접근성이 뛰어나다.

■ Requirement Analysis

● 기능적 요인 (Functional Requirements)

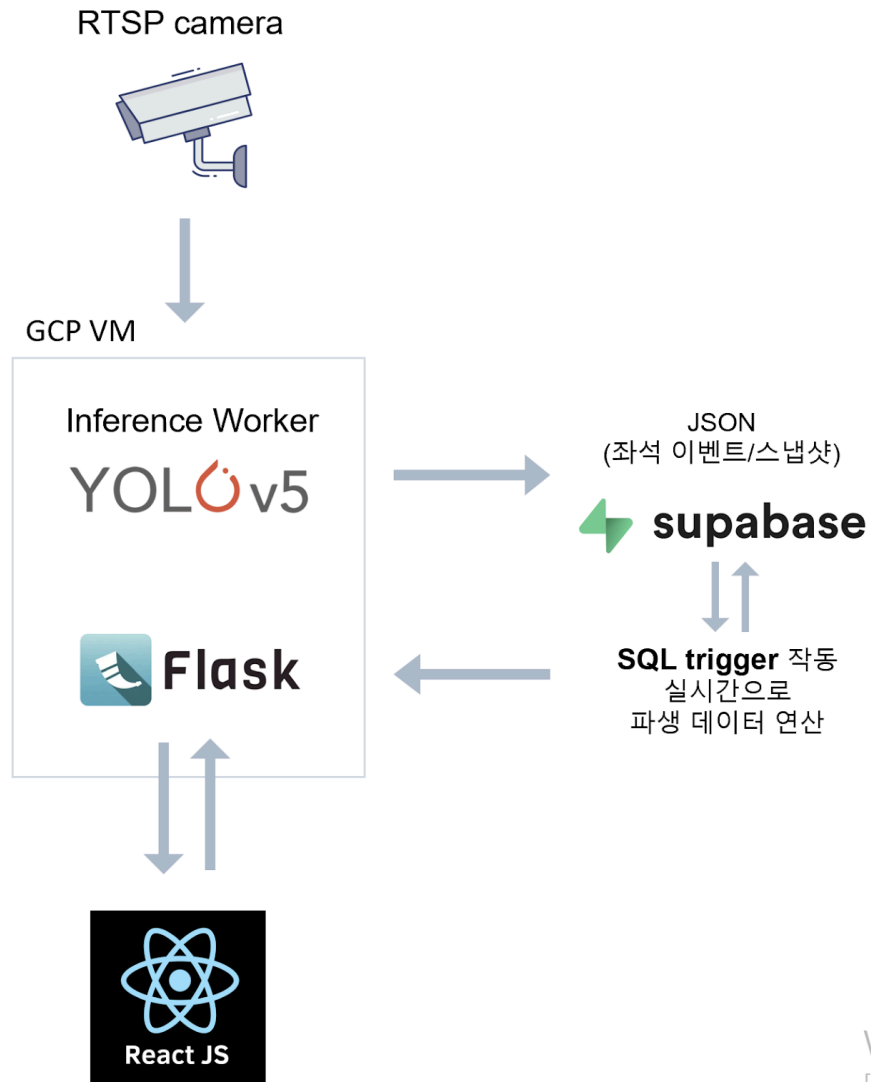
- 좌석 단위 점유 여부 실시간 판별 및 기록 저장
→ 각 좌석의 점유 상태(True/False)를 실시간으로 판별하여 DB에 저장하며 이를 기반으로 혼잡도와 같은 파생 데이터를 산출하여 사용자에게 유용한 정보를 제공한다.
- 실시간 혼잡도 계산 및 순위 제공
→ 특정 공간의 혼잡도를 실시간으로 계산하고, 혼잡도가 낮은 순으로 장소를 제공한다.
- 시간대별 혼잡도 정보 제공
→ 실시간 계산된 혼잡도를 축적하여 시간대별 통계 정보를 제공한다. 이를 통해 사용자는 혼잡도가 낮은 시간대를 파악하고, 공간별 인원 수를 고려한 맞춤형 장소 추천을 받을 수 있다.
- 위치 기반 장소 추천
→ 사용자의 현재 위치를 고려하여 가까우면서도 이용 가능한 공간을 추천한다.

● 비기능적 요인 (Non-functional Requirements)

- 실시간성 보장
 - 감지부터 DB 반영 및 UI 반영까지 전체 지연 시간(latency)을 3초 이내로 유지한다.
 - SQL Trigger를 활용하여 파생 데이터를 실시간으로 연산할 수 있도록 지원한다
- 가용성/확장성
 - 다수의 카메라(n대)에 대응할 수 있도록 수평 확장을 지원한다.
 - 추론, API, DB 모듈을 분리하여 서비스 적용 장소를 손쉽게 확장할 수 있다.
- 보안/프라이버시
 - 서비스 운영 시 영상 데이터는 저장하지 않으며(단, 모델 튜닝 과정은 예외), 네트워크 포트는 최소한만 개방한다(외부: 80/443, 내부: 로컬 사설망).
 - 영상 기반 개인정보는 내부 관리자만 제한적으로 접근 가능하며, 서비스 구동 시에는 영상이 저장되지 않는다.
- 비용 상한
 - 서비스 운영 비용은 VM 사양 및 카메라 대수에 비례하며, 이를 고려한 비용 상한을 설정한다.

■ Service Architecture Design

● 전체 파이프라인



▲ 전체 시스템 아키텍처 구조

웹 주소(학술제 종료 시까지 구동): <https://variable-made-obituaries-borders.trycloudflare.com>

● 주요 설계 포인트

- 이벤트 저장 전략: 실시간 상태 테이블과 세션 이벤트(사용 시작/종료) 테이블 분리(종료 시 1회 기록).
- DB 트리거: detections insert/update 시 혼잡도 집계 테이블/뷰 갱신.
- 실시간 전파: Supabase Realtime으로 프론트가 새로고침 없이 구독/반영.
- 확장성: 추론과 API/DB를 비동기 분리(큐 또는 배치 업서트), 다카메라·다모델 지원.

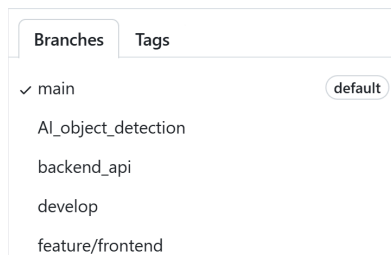
■ Development Process Planning

● 깃 전략

- **main** 브랜치 : 최종 완성물만 올리는 메인 브랜치.
- **develop** 브랜치 : 각자 작업 후 기능을 완성하면 합쳐서 올리는 브랜치.
- **backend_api, feature/frontend, AI_object_detection** 브랜치 : 백엔드, 프론트엔드, ai 파트 팀원이 작업하는 브랜치.

● 협업 방식

- 본 프로젝트에서는 **기능 단위 브랜치 전략**을 적용하였다. 백엔드와 프론트엔드는 각각 개별 작업 브랜치에서 기능을 구현한 뒤, 완성된 기능을 **develop** 브랜치에 병합하였다. 만약 다른 팀원이 **develop** 브랜치에 새로운 기능을 반영한 경우, 각자 최신 내용을 **pull** 받아 이어서 작업을 진행하였다.

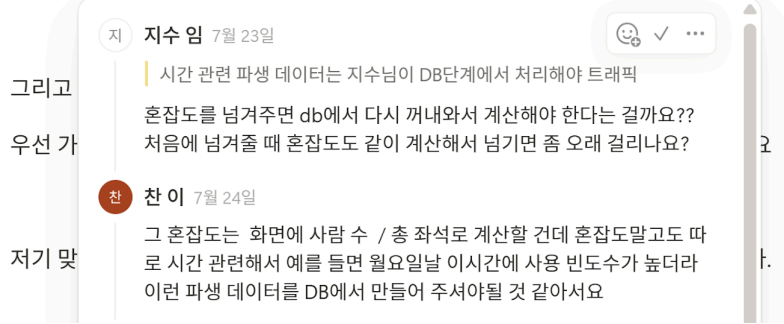


- 특히 AI 파트와의 협업에서는 **실시간 데이터 연동 검증**이 핵심이었다. 데이터가 데이터베이스에 정상적으로 기록되는지, 동시에 웹 화면에도 정확히 반영되는지를 단계별로 확인하며 연결 지점을 점검하였다. 또한 웹 서비스가 일정 수준 구축된 이후에는 AI 모듈과의 실시간 연결을 위해 **Supabase 환경을 확장**하고, SQL Trigger를 활용하여 데이터가 갱신될 때 자동으로 처리되도록 설계하였다.
- 협업 과정에서는 각자의 문제 상황과 진행 상황을 노션(Notion)에 기록하여 공유하였으며, 이를 바탕으로 상호 피드백을 주고받으며 작업을 지속적으로 개선해 나갔다.

Aa 회의 이름	📅 날짜	🔄 진행상태	👥 참석자
학술제 진행 일정	2025년 6월 30일	● 진행 중	
주제 제안서- 교내 공간 추천 서비스	2025년 6월 30일	● 진행 중	
기능 정의	2025년 7월 4일	● 계획 중	
프론트엔드 - ui설계	2025년 7월 4일	● 완료	👤 은우 정
백엔드 - db, api	2025년 7월 21일	● 완료	👤 지 지수 임
ai기능구현	2025년 7월 4일	● 진행 중	👤 안 안성현 🧡 찬 찬 이
협업 도구	2025년 7월 4일	● 계획 중	
학술제 1&2주차- 1차 멘토링 이전 결정할 것	2025년 7월 4일	● 진행 중	
학술제 3주차-1차 멘토링	2025년 7월 4일	● 완료	

좌석 점유가 끝났을 때 DB로 넘길겁니다

그래서 이제 시간 관련 파생 데이터는 지수님이 DB단계에서 처리해야 트래픽이 안 생길 것 같아요



▲ 진행 상황을 공유하며 질의응답과 요구사항을 공유

● 깃허브 디렉토리 구조

```
EmptySpot/
├── backend/                                # Flask 기반 백엔드 서버 코드
│   ├── app.py                            # 메인 서버 애플리케이션 (API 라우팅)
│   ├── requirements.txt                  # 백엔드 의존성 패키지 목록
│   └── .env                             # 환경 변수 (Supabase URL, KEY 등)
├── frontend/                             # React 기반 프론트엔드 코드
│   ├── billgate/
│   │   ├── node_modules/                # 외부 라이브러리 및 의존성 모듈 저장 폴더
│   │   ├── public/                     # 이미지 등 정적 자원 파일 모음
│   │   ├── src/                         # React 실제 소스코드
│   │   │   ├── assets/                  # 프로젝트에서 사용하는 이미지, 아이콘 등 정적 리소스
│   │   │   ├── components/             # 재사용 가능한 공통 UI 컴포넌트 모음
│   │   │   ├── pages/                  # 웹사이트의 각 페이지를 구성하는 컴포넌트 모음
│   │   │   └── App.js                  # React 애플리케이션 엔트리포인트
│   │   └── package.json                 # 프론트엔드 의존성 및 실행 스크립트 정의
├── AI_object_detection/                  # AI 모델 관련 코드 (Object Detection)
│   ├── main.py                          # 실시간 좌석 점유 판단 및 yolo_snapshot.json 업데이트
│   ├── yolo_snapshot.json               # 좌석 상태를 저장하는 JSON 파일
│   ├── json_to_DB_daemon.py            # main.py와 함께 실행되며 JSON 파일을 DB로 전송
│   └── best.pt / custom.pt ...          # YOLOv5 모델 튜닝 가중치 파일
├── supabase/                             # Supabase 관련 설정 파일
│   ├── .temp/                          # Supabase CLI 실행 시 자동 생성된 임시파일
│   └── migrations/                     # DB 스키마 변경 내역 (테이블 생성/수정, 함수·트리거 추가 등)
├── README.md                            # 프로젝트 소개 및 실행 방법
└── .gitignore                           # Git에 포함하지 않을 폴더 설정 (가상환경, Supabase 키 등)
```

■ Data & API Specification

● ERD

- location : 장소의 기본 정보들이 있는 테이블.
 - id (PK) : 장소의 고유 id
 - name, building, campus, capacity, max_group, lat, lng, details : 장소에 대한 메타데이터 (장소 이름, 건물, 학교, 총 좌석, 회의 가능한 인원수, 위도, 경도, 상세정보)
- seat_log : 좌석 점유에 변경사항이 있을 때 좌석별 점유율이 업데이트 되는 테이블.
 - id (PK) : 좌석 로그의 기본 인덱스
 - location_id (-> location.id FK) : 장소의 고유 id (location 테이블을 참조)
 - occupied : 좌석 점유 중인지 1, 0으로 표현
 - start_time, end_time : 좌석 점유가 시작하고 끝나는 지점
 - seat_id : 좌석 id
- congestion_snapshot : 점유 변경 사항에 따라 실시간으로 혼잡도를 계산한 테이블.
 - id (PK) : snapshot 기본 인덱스
 - location_id (-> location.id FK) : 장소의 고유 id (location 테이블을 참조)
 - current_people : 현재 location_id 에 해당하는 장소의 점유 인원수
 - congestion : 현재 location_id 에 해당하는 장소의 혼잡도
 - date, hour, day_week, timestamp : 업데이트 된 혼잡도의 시간대 기록
- congestion_hourly : 시간대별 혼잡도 계산을 위한 파생 테이블.
 - location_id (-> location.id FK), date, hour (복합 PK) : 시간대별 혼잡도를 조회하기 위해 복합키 사용
 - samples : 각 날짜, 시간대에 해당하는 업데이트 된 혼잡도 데이터의 개수
 - sum_congestion, sum_people : 업데이트 된 혼잡도와 점유 인원수 총 합
(이후 시간대별 평균 혼잡도 계산을 할 때 sum_congestion / samples 연산을 한다)
 - updated_at : 업데이트 된 시간대
- latest_congestion, v_hourly_avg_28d, v_location_congestion (뷰)
: 실시간으로 변하는 좌석 점유 데이터를 빠르게 계산하여 혼잡도를 보여주기 위해 뷰를 사용

● 테이블 간 관계

- seat_log -> congestion_snapshot : 좌석 로그 데이터가 업데이트 되면 SQL trigger를 통해 실시간으로 혼잡도 스냅샷이 생성된다.
- congestion_snapshot -> congestion_hourly : snapshot 테이블 역시 업데이트 되면 SQL trigger를 통해 실시간으로 개별 시점 데이터를 시간 단위로 집계한다.
- location <-> seat_log / congestion_snapshot / congestion_hourly : 모든 혼잡도 정보는 location 테이블의 id를 참조한다.

- latest_congestion, v_hourly_avg_28d, v_location_congestion (뷰) : 파생데이터가 업데이트 될 때 가장 최근 데이터 값으로 갱신되며 혼잡도 관련 값을 요청 받을 때 바로 조회된다.



▲ ERD 설계도

● SQL Trigger

- trg_snapshot_on_seat_log
 - seat_log 테이블에서 좌석 점유 상태(occupied, start_time, end_time) 가 변경될 때 마다 실행되는 함수이다.
 - **혼잡도 기록** : 먼저 해당 장소의 총 좌석수를 조회하고, 점유 중인 좌석 수를 계산하여 현재 이용 인원을 저장한다. 그 후 현재 인원 / 총 좌석 수 계산을 한 뒤, congestion_snapshot 테이블에 기록한다.
 - **뷰 생성** : latest_congestion 뷰를 정의하여 가장 최근 혼잡도 조회할 수 있게하고, location 테이블과 latest_congestion 테이블을 조인하여 api에서 바로 사용할 수 있도록 v_location_congestion 뷰를 생성한다.

- `trg_congestion_hourly_rollup`
 - `congestion_snapshot`에 새로운 데이터가 삽입될 때 실행되는 함수이다.
 - **시간대별 혼잡도 집계** : 새로 들어온 스냅샷의 `location_id`, `date`, `hour`(복합PK)를 기준으로 `congestion_hourly` 테이블에 데이터를 누적시킨다. 해당 시간대가 없으면 새로 생성하고, 존재하면 샘플 개수와 혼잡도 총합을 증가시켜 집계한다.
 - **뷰 생성** : 최근 28일간의 시간대별 혼잡도를 평균 내어 제공하는 `v_hourly_avg_28d` 뷰를 생성한다. Api나 리포트에서 시간대별 평균 혼잡도를 조회할 때 활용한다.

● API 명세

☰ index	Aa 기능	</> method	📄 api path
메인 페이지	</> 위치 기반 공간 추천	GET	/api/recommendation/location?lat=값&lng=값
	</> 혼잡도 기반 공간 추천	GET	/api/recommendation/congestion
장소 소개	</> 장소의 상세 정보 조회	GET	/api/places/{place_id}
	</> 장소별 시간대 인원수 조회	GET	/api/congestion/avg?location_id=값&hour=값
인원별 장소 추천	</> 인원별 장소 추천 (혼잡도 낮은순)	GET	/api/congestion_people?group=값

- **위치 기반 공간 추천** : 웹에서 사용자 위치(lat,lng)를 받고, db에서 모든 장소의 lat, lng를 불러온다. 각 장소와 사용자의 거리를 Haversine 공식으로 계산하여 거리가 가까운 순서대로 정렬하여 반환한다.
- **혼잡도 기반 공간 추천** : `v_location_congestion` 뷰에서 현재 인원과 혼잡도 정보를 조회한다. 혼잡도가 낮은 순으로 정렬하여 반환한다.
- **장소의 상세정보 조회** : 전달받은 `place_id` 값과 일치하는 장소를 조회하여 상세 정보를 반환한다.
- **장소별 시간대 인원수 조회** : 최근 28일 기준 집계 뷰(`v_hourly_avg_28d`)에서 특정 장소의 특정 시간대 평균 혼잡도와 인원수를 조회해 반환한다.
- **인원별 장소 추천 (혼잡도 낮은순)** : `v_location_congestion` 뷰에서 최대 수용 인원이 요청 인원(group) 이상인 장소만 추려, 혼잡도가 낮은 순으로 추천 목록을 반환한다.

■ Security & Deployment Setup

● Security

- **네트워크/프록시** : Nginx 리버스 프록시, HTTP→HTTPS 리다이렉트, HSTS.
- **RLS** : 모든 원본/집계 함수는 RLS on. api 요청을 받을 때 데이터를 json 형태로 보내기 때문에 클라이언트는 접근 불가하게 설정하였다.
- **CORS** : 학술제 시연 편의를 위해 CORS는 전체(*) 허용으로 설정하였다. 실제 서비스 운영 시에는 보안을 위해 고정 도메인만 허용하는 화이트리스트 방식으로 전환할 예정이다.
- **방화벽** : SSH/HTTP/HTTPS 등 최소 포트만 오픈 , 외부 포트는 80/443만 공개(80→443 리다이렉트), 내부 앱 포트(예: 8000)는 비공개

☐	default-allow-http	Ingress	http-server	IP ranges:	tcp:80	Allow	1000	default
☐	default-allow-https	Ingress	https-server	IP ranges:	tcp:443	Allow	1000	default

- **VPC 방화벽**: 외부는 **tcp:80, 443**만 허용(기타 포트 차단)

Firewalls

- ☒ Allow HTTP traffic
- ☒ Allow HTTPS traffic
- ☐ Allow Load Balancer Health checks

Network tags

Network tags

api-server X http-server X https-server X ?

- **VM 네트워크 태그**: **http-server, https-server** 적용(웹 트래픽 허용)

● Deployment Setup

- 배포 환경: GCP Compute Engine VM(Ubuntu)에 Flask 기반 API를 Gunicorn으로 구동, Nginx가 80/443을 수신(TLS 종료·리버스 프록시)
- 배포: systemd(or tmux)로 API 지속 실행, 로그 로테이션
- 모니터링: Nginx/Gunicorn 로그, 추론 지연, DB 트리거 오류 캡처
- 백업/마이그레이션: 주기적 스냅샷, 스키마 버전 관리(Alembic 등)
- 비용/스케일: 카메라 수↑ → 추론 워커 수 확장(멀티 프로세스/노드로 처리량 선형 증가)

■ AI Feature Specification

Part 1. AI Feature Specification

1. 시스템 개요 (System Overview)

본 시스템은 고정된 카메라 영상을 실시간으로 분석하여 각 좌석의 점유 상태를 판단하고, 공간 전체의 혼잡도를 수치화하여 제공하는 AI 비전 기반 좌석 점유 관리 솔루션이다.

시스템의 기반이 되는 멘토링을 통해 교수님에게서 받은 오픈소스 AI 모델(Jetson Nano 기반 YOLO)은 객체 탐지 성능이 우수하지만, 실제 좌석 관리 환경에 그대로 적용하기에는 명확한 한계가 있다. 오픈소스 모델은 시간의 흐름을 고려하지 않고 매 순간의 정지된 이미지(프레임)만을 기준으로 객체 유무를 판단합니다. 이 때문에 좌석 앞을 잠시 스쳐 지나가는 사람, 주위 환경 같은 객체에 포함되지 않는 물체까지 모두 '점유' 상태로 잘못 인식하여 데이터의 신뢰도를 심각하게 저하시키는 문제가 발생했다.

우리는 이 문제를 해결하기 위해, 단순히 객체를 감지하는 것을 넘어 일정 시간 이상 좌석에 '지속적으로' 머무르는 경우에만 실제 '점유'로 확정하는 독자적인 시간 기반 판정 로직을 추가했다. 이를 통해 순간적인 움직임이나 일시적인 사물로 인한 오탐지를 효과적으로 걸러내어 서비스의 정확도를 획기적으로 높였다.

나아가, 본 시스템의 차별점은 '사람 겹침 및 가림(Occlusion)' 상황에 특화하여 자체 학습시킨 커스텀 AI 모델에 있다. 이를 통해 사람들이 밀집되어 서로를 가리는 복잡한 환경에서도 기존 시스템이 빈번히 실패하는 개별 인원 식별을 정밀하게 수행한다.

2. 핵심 기능 (Key Features)

2.1. 좌석 단위 개별 모니터링 (Individual Seat Monitoring):

카메라 영상 내 각 좌석 영역을 개별적인 관심 영역(ROI)으로 설정하여, 수십 개의 좌석 상태를 동시에 독립적으로 추적하고 관리한다.

2.2 시간 기반의 지능형 점유 판정 (Time-based Intelligent Occupancy Judgment):

점유 확정 (Occupancy Confirmation): 특정 좌석 ROI 내에서 '사람(person)' 클래스는 물론, 사람이 앉아있거나 소지품이 놓인 상태를 학습한 'chairfull' 및 'object' 클래스가 10초 이상 연속으로 감지될 경우에만 해당 좌석을 '점유(ON)' 상태로 확정합니다. 이를 통해 단순히 좌석 옆을 지나가는 유동인구를 점유로 오인하는 것을 방지합니다.

이탈 확정 (Vacancy Confirmation): '점유' 상태의 좌석에서 위에서 언급된 점유 관련 클래스(person, chairfull, object)의 객체가 5초 이상 연속으로 미감지될 경우 '비어있음(OFF)' 상태로 전환합니다. 이는 사용자가 잠시 가려지거나 모델이 순간적으로 놓치는 상황에도 상태를 안정적으로 유지하는 강건성을 제공합니다.

2.3 점유 시간 자동 기록 (Occupancy Duration Logging):

각 좌석이 '점유(ON)' 상태로 전환된 시점과 '비어있음(OFF)' 상태로 전환된 시점을 자동으로 기록하여, 좌석별 총 사용 시간 및 이용 패턴 분석을 위한 데이터를 생성한다.

2.4 실시간 전체 혼잡도 분석 (Real-time Congestion Analysis):

개별 좌석 상태와 별개로, 카메라 프레임 내에 감지되는 전체 사람 수를 실시간으로 카운트하여 공간의 전반적인 혼잡도 또는 밀집도를 직관적인 수치로 제공한다.

3. 시스템 아키텍처 및 데이터 인터페이스 (System Architecture & Data Interface)

3.1 시스템 아키텍처(System Architecture)

실시간 카메라 피드 (RTSP) → 처리 장치 (pc/노트북) → YOLOv5 객체 탐지 → 지능형 점유 판정 로직 → 데이터 생성 및 데이터베이스 전송

3.2 입력 데이터 (Input Data)

- 종류: 실시간 비디오 스트림
- 소스: RTSP 프로토콜을 지원하는 IP 카메라
- 해상도: 1280 x 720

3.3 출력 데이터 (Output Data)

- 실시간 상태 스냅샷 (JSON 형식)
- 목적: 현재 시점의 모든 좌석 상태와 전체 혼잡도를 웹 대시보드 등에 실시간으로 표시하기 위함.
- 내용: 모든 좌석(ROI)의 현재 점유 상태(0 또는 1), 점유 시작 시각, 그리고 전체 사람 수를 포함.

Part 2. 프로젝트 개발 배경 및 과정

4. 프로젝트 배경 및 동기 (Project Background & Motivation)

4.1. 초기 접근 및 한계 봉착

본 프로젝트는 실시간 객체 탐지를 목표로, 대표적인 알고리즘인 YOLO(You Only Look Once)를 기반으로 시작되었다. 초기 단계에서는 기본 YOLO 모델과 Roboflow 공개 데이터셋을 활용한 커스텀 학습을 시도했으나, 처리 지연(delay) 문제와 훈련 데이터와 실제 적용 환경 간의 차이로 인해 유의미한 성능을 확보하지 못하는 한계에 부딪혔다.

4.2. 핵심 문제 발견

이러한 시행착오 이후, 멘토링을 통해 전달받은 YOLO 시스템을 새로운 출발점으로 삼았다. 이 시스템은 단일 객체나 분리된 객체에 대해서는 준수한 탐지 성능을 보였으나, 실제 환경과 유사한 복잡한 시나리오에 적용했을 때 치명적인 문제점을 발견했다.

핵심 문제점: 다수의 사람이 서로 겹쳐 있거나 신체 일부가 다른 사물에 의해 가려지는(Occlusion) 상황에서 모델의 객체 탐지율이 급격하게 저하되었다. 많은 경우, 겹쳐 있는 여러 사람을 하나의 객체로 잘못 인식하거나 아예 탐지하지 못하는 현상이 빈번하게 발생했다.

5. 문제 해결을 위한 개발 전략 (Development Strategy for Problem Solving)

5.1. 목표 설정

앞서 발견한 '겹침 및 가림' 상황에서의 탐지 실패 문제를 해결하는 것을 프로젝트의 핵심 목표로 설정했다. 이를 통해 어떤 복잡한 상황에서도 강건한(Robust) 탐지 성능을 확보하고자 했다.

5.2. 데이터 중심 접근법 (Data-Centric Approach)

문제 해결을 위해 복잡한 모델 구조를 변경하기보다, 문제의 근본 원인인 '특정 상황 데이터의 부재'를 해결하는 것이 더 효율적이고 직접적이라 판단했다. 따라서 문제 상황을 가장 잘 대표하는 고품질의 커스텀 데이터셋을 구축하여 기존 모델을 파인튜닝하는 데이터 중심의 접근법을 채택했다.

6. 모델 개발 상세 과정 (Model Development Details)

6.1. 겹침에 특화된 데이터셋 구축

데이터 수집 및 선별: 약 1시간 분량의 일상생활 영상에서 5초 간격으로 프레임을 캡처했다. 이 중, 모델이 가장 취약점을 보이는 다양한 겹침 및 가림(occlusion) 상황을 대표하는 고품질 이미지 50장을 최종 선별했다. 이는 양보다 질에 초점을 맞춰, 제한된 자원 내에서 모델 성능을 효율적으로 개선하기 위함이었다.

데이터 라벨링: 라벨링 작업의 효율성과 정확성을 높이기 위해 Roboflow 플랫폼을 활용한 반자동(Semi-automatic) 방식을 채택했다.

- **1차 자동 라벨링:** Roboflow의 사전 학습 모델로 초기 바운딩 박스를 자동 생성하여 작업 시간을 단축했다.
- **2차 수동 검수 및 보완:** AI가 생성한 결과를 사람이 직접 검수하며 오탐지를 수정하고, 특히 모델이 놓친 겹쳐 있거나 가려진 사람에 대해 수동으로 정밀하게 바운딩 박스를 추가하여 데이터의 품질을 확보했다.

6.2. 모델 파인튜닝 (Model Fine-tuning)

- **기본 모델 (Base Model)**
 - **모델 아키텍처:** YOLOv5
 - **초기 가중치:** 멘토링을 통해 받은 1차 파인튜닝된 가중치를 초기값으로 사용하여, 이전 학습 지식을 계승하고 학습 효율을 높였다.
- **학습 환경 (Training Environment)**
 - **플랫폼:** Google Colaboratory (Colab)
 - **GPU:** NVIDIA T4
- **주요 하이퍼파라미터 (Key Hyperparameters)**
 - **Epochs:** 100
 - **Batch Size:** 16
 - **Learning Rate:** 0.01

학습 진행: 앞서 구축한 '사람 겹침' 특화 데이터셋을 이용하여 기반 모델에 대한 추가 학습(파인튜닝)을 수행했다. 이를 통해 모델이 복잡한 겹침 상황에 대한 이해도를 높이고 해당 시나리오에 대한 전문성을 갖추도록 했다.

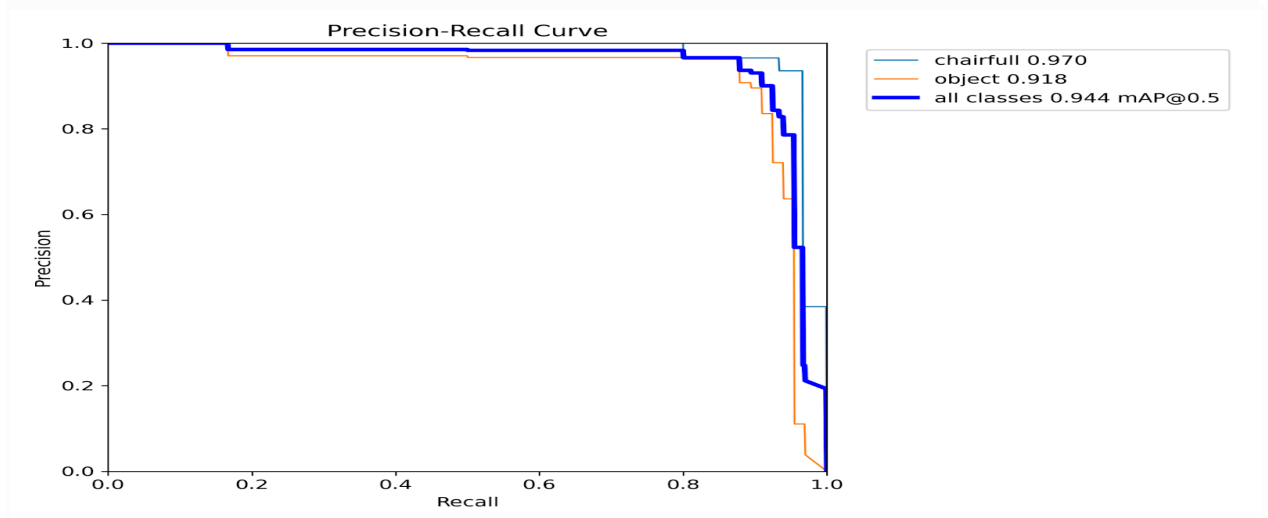
7. 성능 검증 (Performance Verification)

본 시스템의 성능 검증은 시스템의 핵심인 커스텀 AI 모델의 기술적 완성도를 정량적으로 분석하고, 이를 바탕으로 실제 문제 상황에서 기존 모델 대비 우수성을 정성적으로 비교하는 두 단계로 진행되었다. 정량적 분석은 모델의 예측 정확도를 나타내는 핵심 지표(mAP, F1-Score)를 통해, 정성적 분석은 시나리오 기반의 시각적 비교를 통해 수행하여 시스템의 종합적인 성능과 신뢰성을 입증하고자 한다.

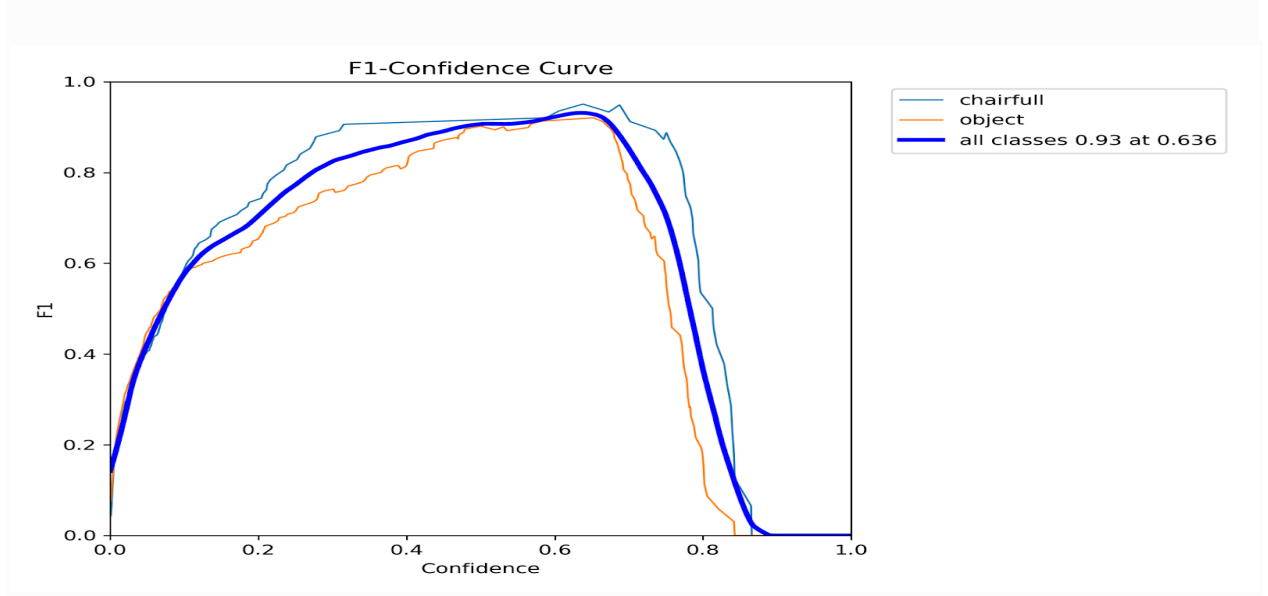
7.1. AI 모델의 정량적 성능 검증

본 시스템은 점유 상태를 정확히 판단하기 위해 '사람이 앉아있는 의자(chairfull)', '소지품(object)' 등 커스텀 클래스를 정의하여 AI 모델을 학습시켰다. 모델의 성능은 정밀도(Precision)와 재현율(Recall)의 관계를 나타내는 PR Curve와 F1-Score를 통해 객관적으로 평가하였다.

[그림 7-1] Precision-Recall (PR) Curve



[그림 7-2] F1-Confidence Curve



- 클래스별 높은 탐지 정확도(mAP) 달성

[그림 7-1]의 PR Curve는 그래프가 우측 상단에 근접한 이상적인 형태로, 모델이 매우 높은 성능을 보임을 나타낸다. 전체 클래스에 대한 mAP(mean Average Precision) @0.5 점수는 0.944를 기록했으며, 이는 약 94.4%의 높은 평균 정밀도로 객체를 탐지함을 의미한다. 특히, 핵심 커스텀 클래스인 'chairfull'은 0.970, 'object'는 0.918이라는 개별 mAP를 달성하여, 사람이 앉아있는 상태와 소지품으로 점유된 상태를 매우 정밀하게 구분해내는 특화된 성능을 입증하였다.

- 최적화된 예측 신뢰도(F1-Score)

[그림 7-2]는 정밀도와 재현율의 조화 평균인 F1-Score를 나타낸다. 본 모델은 최대 0.93이라는 높은 F1-Score를 기록하였다. 이는 모델의 예측 신뢰도(Confidence) 임계값을 약 0.636으로 설정했을 때, 잘못된 탐지(오탐)와 탐지 누락(미탐)을 모두 최소화하는 최적의 균형점을 확보했음을 의미하며, 시스템 운영의 안정성을 뒷받침한다.

7.2. 시나리오 기반 정성적 성능 비교

정량적으로 입증된 모델의 우수성이 실제 문제 해결에 어떻게 기여하는지 확인하기 위해, 기존 범용 모델이 빈번히 실패하는 핵심 시나리오를 선정하여 본 시스템의 성능과 비교하였다.

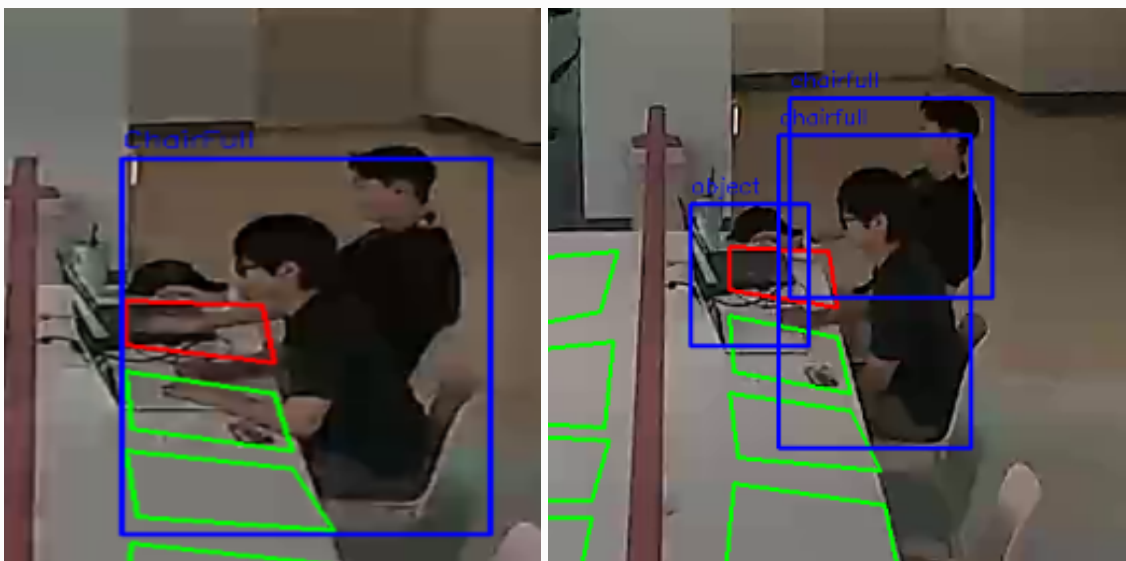
시나리오 1: 단순 통행으로 인한 점유 오탐

상황: 사용자가 비어있는 좌석 근처를 지나가는 경우.

분석: 기존 모델은 순간적으로 포착된 사람을 '점유'로 오인한다. 반면, 본 시스템은 시간 기반 점유 판정 로직을 통해 단기적인 움직임을 필터링하여 '비어있음' 상태를 안정적으로 유지한다.

시나리오 2: 사람 겹침 및 가림(Occlusion)으로 인한 미탐

상황: 점유된 좌석의 사용자가 다른 통행인에 의해 일시적으로 가려지는 경우.

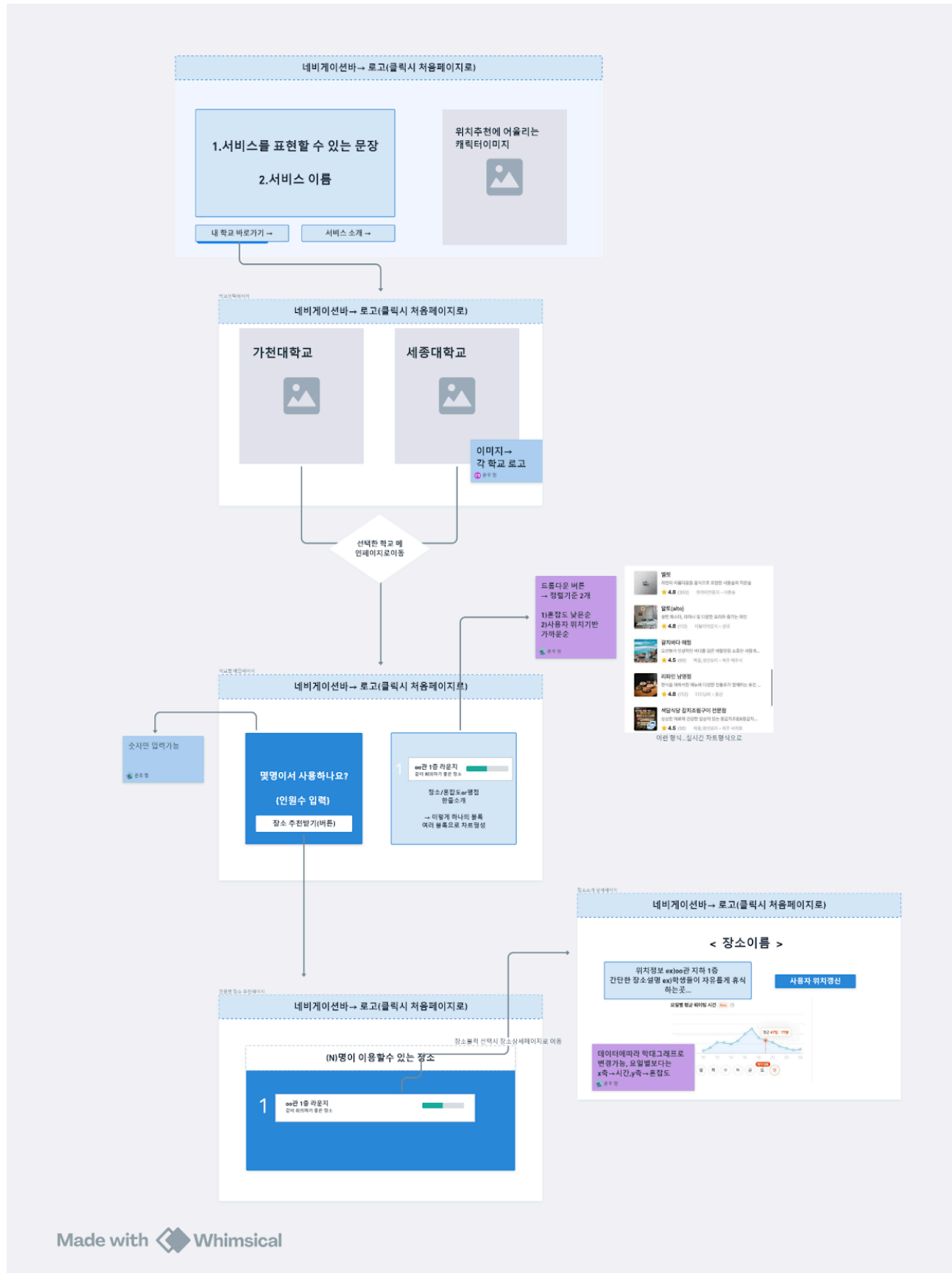


분석: 기존 모델은 가려진 사용자를 탐지하지 못해 '비어있음'으로 오판한다. 반면, Occlusion에 특화 학습된 본 시스템의 커스텀 모델(mAP 0.970의 'chairfull' 클래스)은 가려진 상황에서도 객체 정보를 강건하게 유지하여 '점유' 상태를 정확하게 지속한다.

■ Wireframe & User Flow Design

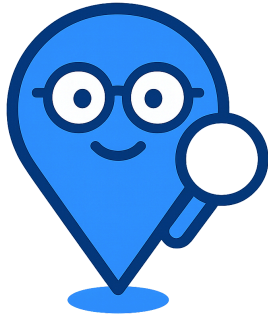
● 사용자 인터페이스 설계

○ UI설계도



○ **웹서비스 로고 및 캐릭터**

- ‘위치를 찾는다’ 는 느낌을 받을 수 있는 느낌의 간단한 이미지와 버튼,로고등에 사용된 파란색을 유사하게 사용하여 통일감을 줌.



timebillgates

○ **컬러 테마**

- 웹 페이지 디자인 시 사용한 컬러

#007bff	#4a90e2
	

■ 웹 서비스 실행 가이드

● 웹 서비스 URL

- <https://variable-made-obituaries-borders.trycloudflare.com/>

● 웹 서비스 실행 방법 (로컬 환경 기준)

- 가상환경 생성 및 활성화
- `pip install -r requirements.txt` 명령어를 통해 백엔드 의존성 패키지를 설치
- `cd backend` 명령어로 폴더 경로를 이동한 뒤, `flask run`을 실행
- React 설치
- 새로운 터미널을 열어 `cd frontend` → `cd billgate` 순으로 경로를 이동한다.
- `npm start` 명령어를 실행하여 프론트엔드를 구동한다.

● ai 실행 방법

- yolo 모델 설치 : <https://github.com/ultralytics/yolov5>
- `pip install -r requirements.txt` 명령어를 통해 모델 실행에 필요한 패키지를 설치

● 깃허브

- <https://github.com/imudi00/EmptySpot>

● 참고 문서

- <https://github.com/Artecrowd/yolov5>